

COSC264

Introduction to Computer Networks and the Internet

Introduction to Routing

Dr Barry Wu

Wireless Research Centre (WRC)

University of Canterbury

Barry.wu@canterbury.ac.nz

Learning Objectives

- At the end of the lectures, you will be able to:
 - Define the term routing
 - Describe the term autonomous system
 - Understand the difference between forwarding and routing
 - Understand the principal routing strategies
 - Discuss approaches to routing
 - Compare and contrast the two representative routing algorithms

Outline

- Routing overview
- Link state routing (Dijkstra's algorithm)
- Distance vector routing (Bellman-Ford algorithm)
- Summary

Routing Overview

What is routing?

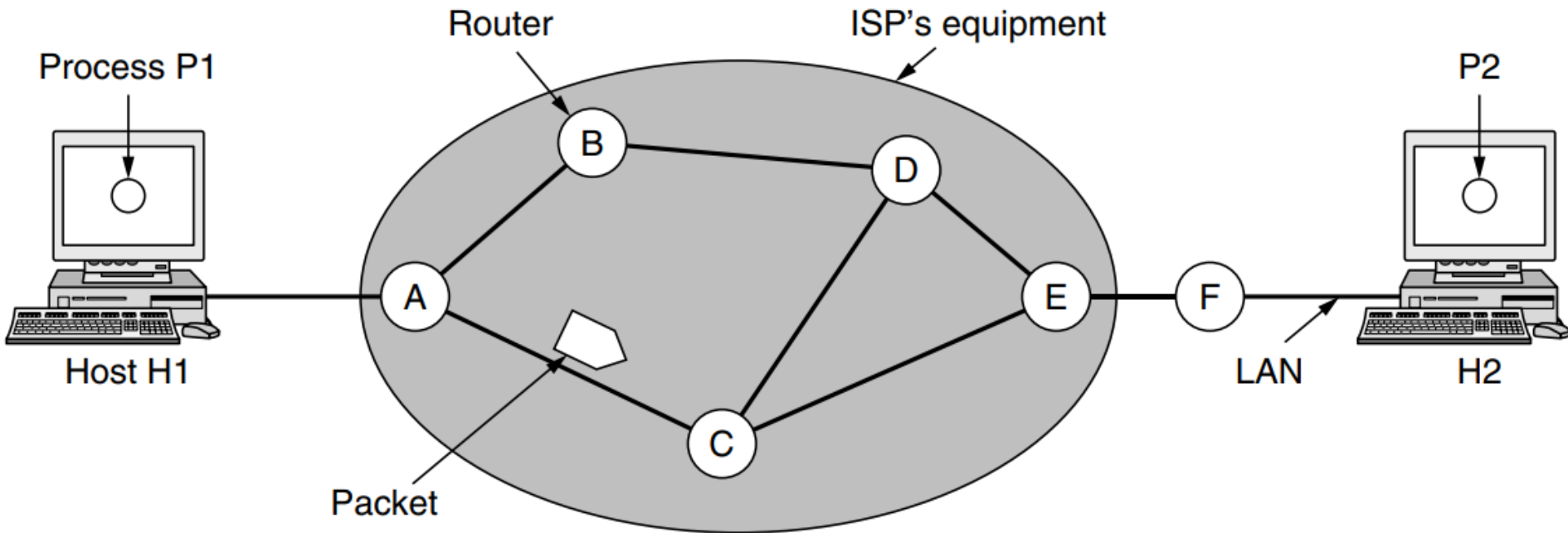
- A famous quotation from RFC 791 Internet Protocol

“A name indicates what we seek.
An address indicates where it is.
A route indicates how we get there.”
-- Jon Postel
- Routing refers to the network-wide process that determines the end-to-end paths that packets take from source to destination.



How does routing occur?

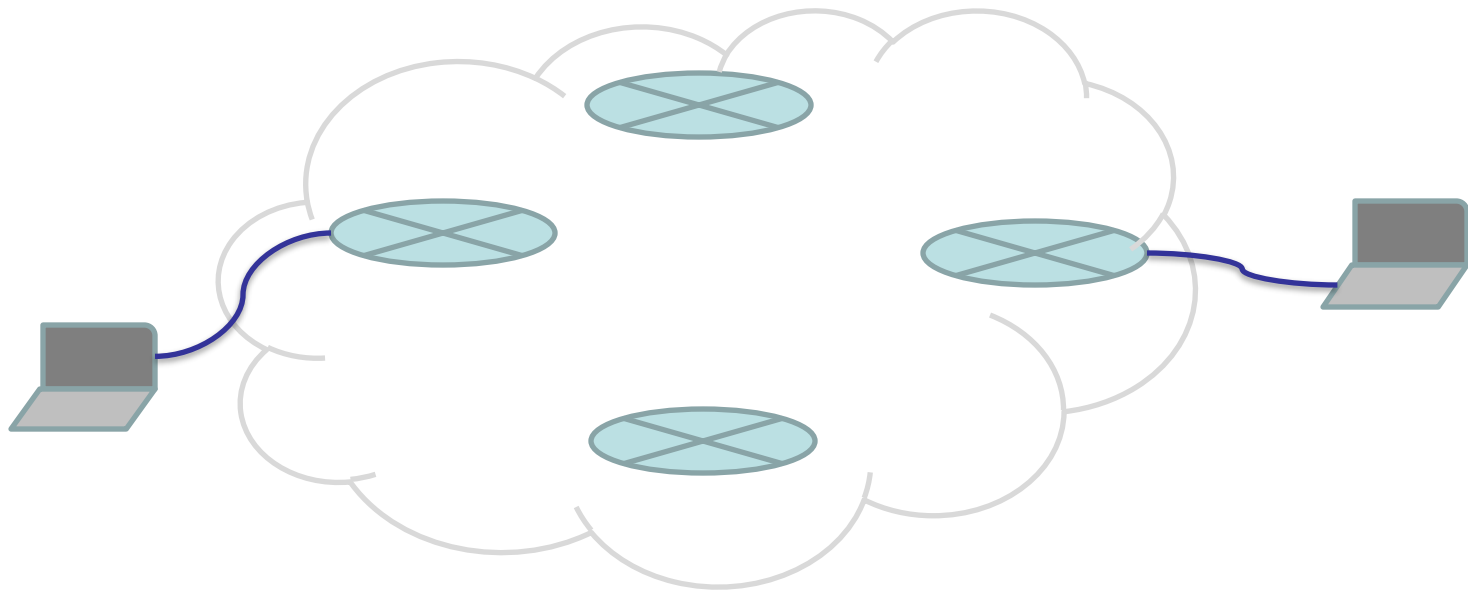
- The environment of the network layer protocols



Source: Andrew S. Tanenbaum & David J. Wetherall, Computer Networks, 5th edition, 2010, Figure 5-1.

Why does routing matter?

- The network needs to work out **the path** from the sender to the receiver *automatically* in a dynamic environment.



Autonomous System (AS)

An AS exhibits the following characteristics:

- is a set of routers and networks managed by a *single* organisation
- typically consists of a group of routers exchanging information via a *common* routing protocol
- is *connected*, except in times of failure

AS Examples

- Each AS is assigned an autonomous system number (ASN)
- Typically each Internet Service Provider (ISP) is an AS

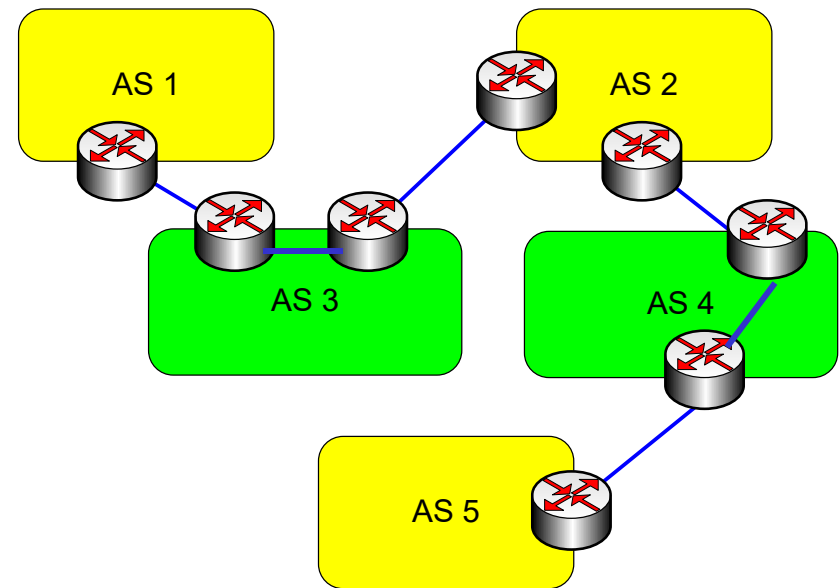


- Other AS examples: corporations, universities

Common Types of ASs

ASs are categorised depending on the way they are connected.

- Stub AS
 - is only connected to another AS
 - e.g., AS1, AS5
- Multi-homed AS
 - has more than one connection to other ASs, but **does not allow** traffic to pass through
 - e.g., AS2
- Transit AS
 - is connected to more than one AS and **allows** the traffic to pass through
 - e.g., AS3, AS4

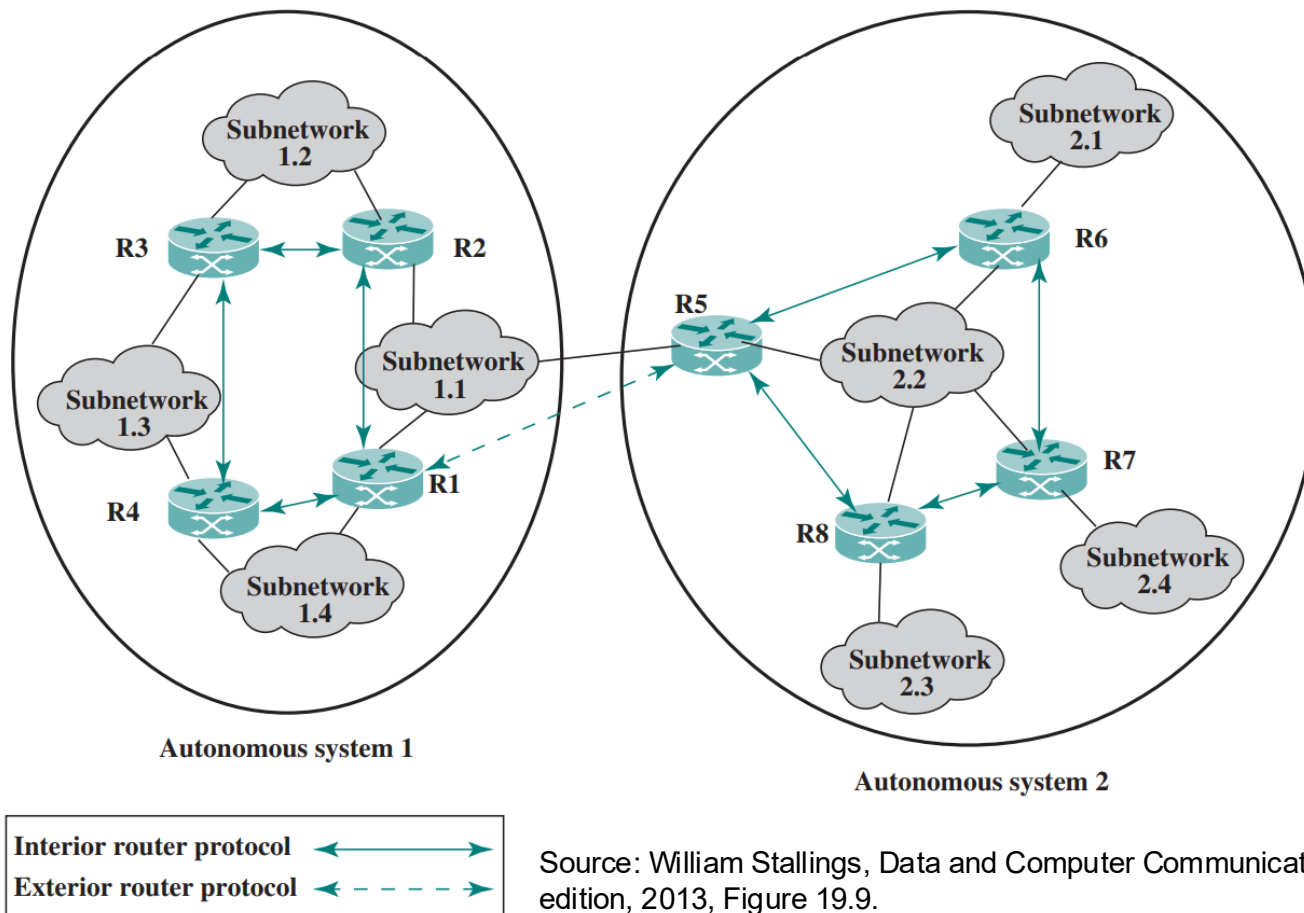


ASN Lookup

University of Canterbury	
AS Handle	AS9432
ASN Name	CANTERBURY-AS
Organization Name	University of Canterbury
Organization ID	ORG-VNZL1-AP-APNIC
Country	 New Zealand
Regional Registry	APNIC
IPv4 CIDRs	• 132.181.0.0/16 • 202.36.178.0/23

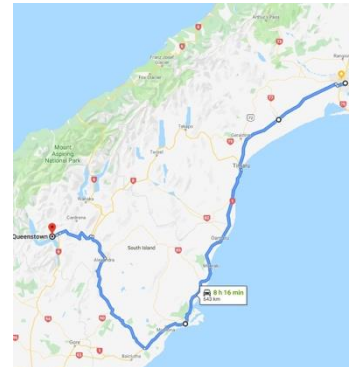
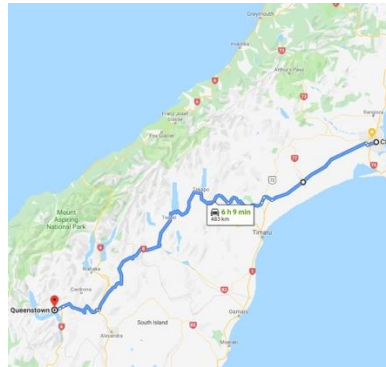
Exterior and Interior Routing Protocols

- Interior router protocol **within AS**
- Exterior router protocol for **AS-to-AS** routing



Routing vs. Forwarding

- Routing: determine which paths to use
 - Routing algorithms calculate these paths



- Forwarding: handle each packet as it arrives
 - The router consults a forwarding table to decide which output link to send the packet



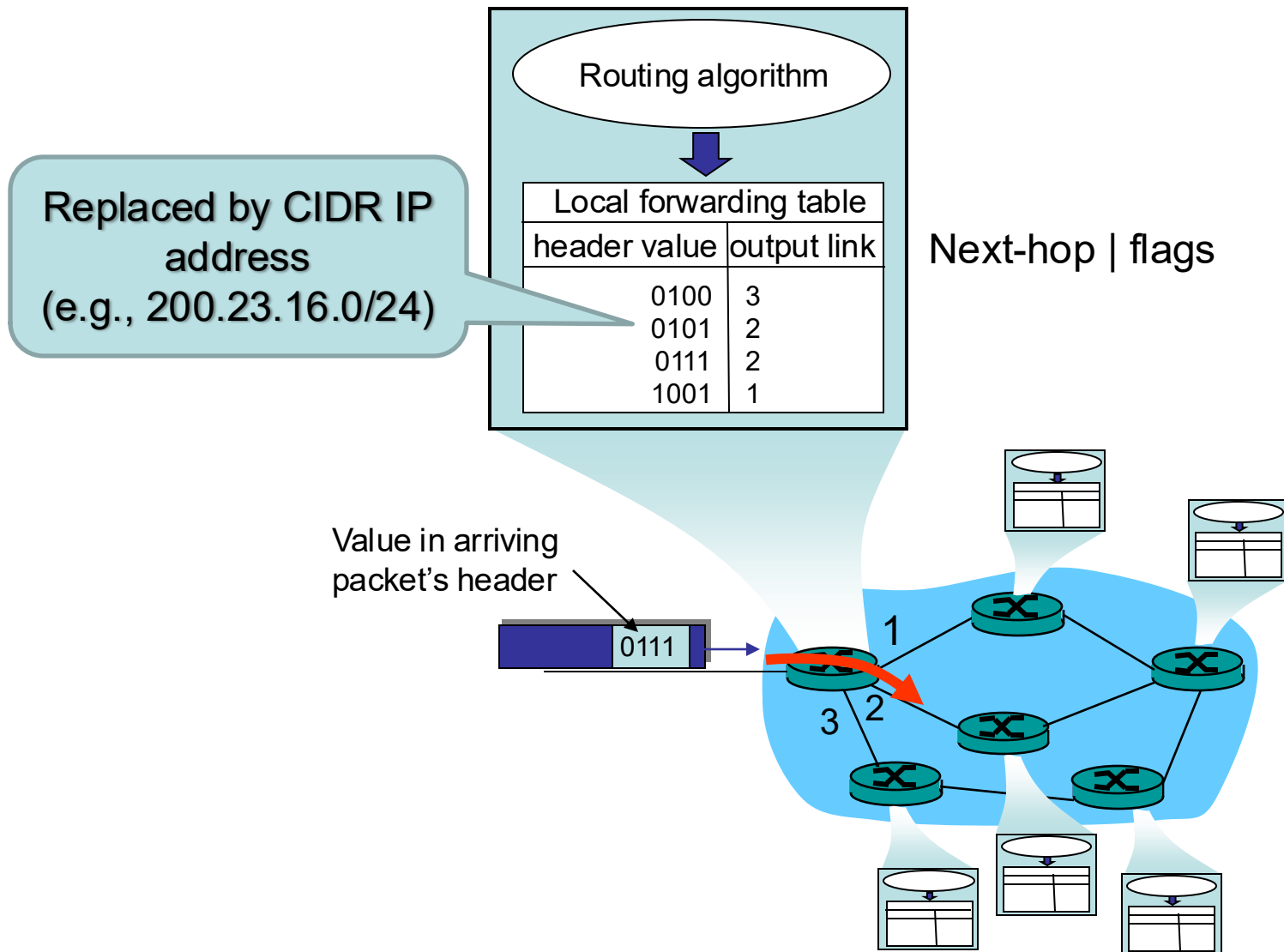
Routing vs. Forwarding (cont.)

- Routing: computing paths the packets will follow
 - routers talking amongst themselves and jointly *creating forwarding tables*
 - **non-real time**: latency up to 2 minutes
- Forwarding: directing every data packet to an outgoing link
 - individual router *using a forwarding table*
 - **real time**; may be implemented in specialised **hardware**

Forwarding Table

- A forwarding table:
 - results from the execution of the **routing protocol** (dynamic routing, or static / preconfigured routing)
 - is consulted for every packet
 - is changed on relatively large timescales, e.g., upon topology changes, load changes or changes in metrics
- At minimum, this contains the **interface identifier and next hop information** for each reachable destination network prefix.

Routing and Forwarding



Q: What is the difference between routing algorithms and routing protocols?

Routing Algorithms and Routing Protocols

- A ***routing algorithm*** solves a routing problem with **ideal assumptions**.
- A ***routing protocol***
 - embeds a *routing algorithm* into **a real networking context**:
 - o It operates in a distributed environment
 - o It incorporates explicit information exchange among nodes
 - o Information exchange takes time and might fail, the protocol must consider these possibilities

Routing Strategies

Fixed/static routing

- A single, permanent route is configured for each source-destination pair of nodes in the network.
- The routes are fixed, or at least only change when there is a change in the topology of the network.

Flooding

- A packet is sent by a source node to every one of its neighbours.
- At each node, an incoming packet is retransmitted on all outgoing links except for the link on which it arrived.

Adaptive/dynamic routing

- Routing decisions change as conditions on the network change, such as failure or congestion.
- Information about the state of the network must be exchanged among the nodes.

Random routing

- A node selects only one outgoing path for retransmission of an incoming packet.
- The outgoing link is chosen at random, excluding the link on which the packet arrived.

Approaches to Routing

Distance-vector routing

Each router maintains a vector giving the best-known distance to each destination and which link to use to get there.

These vectors are updated by exchanging information with the neighbours.

Each router builds up **a routing database based on neighbour updates.**

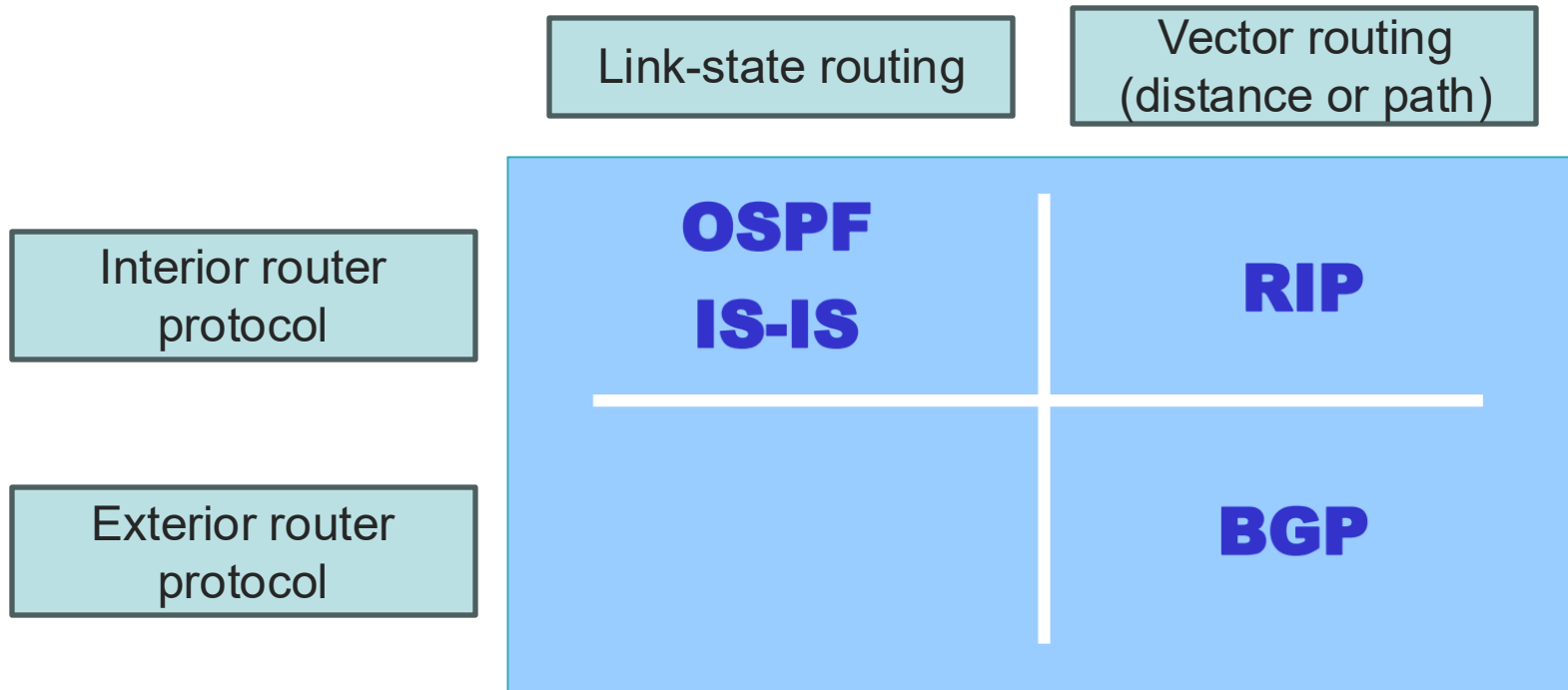
Link-state routing

When a router is initialised, it determines the link cost on each of its network interfaces.

The router advertises this set of link costs to **all other routers** in the internet topology.

Each router builds up **the complete topology** of the configuration and then performs a routing calculation.

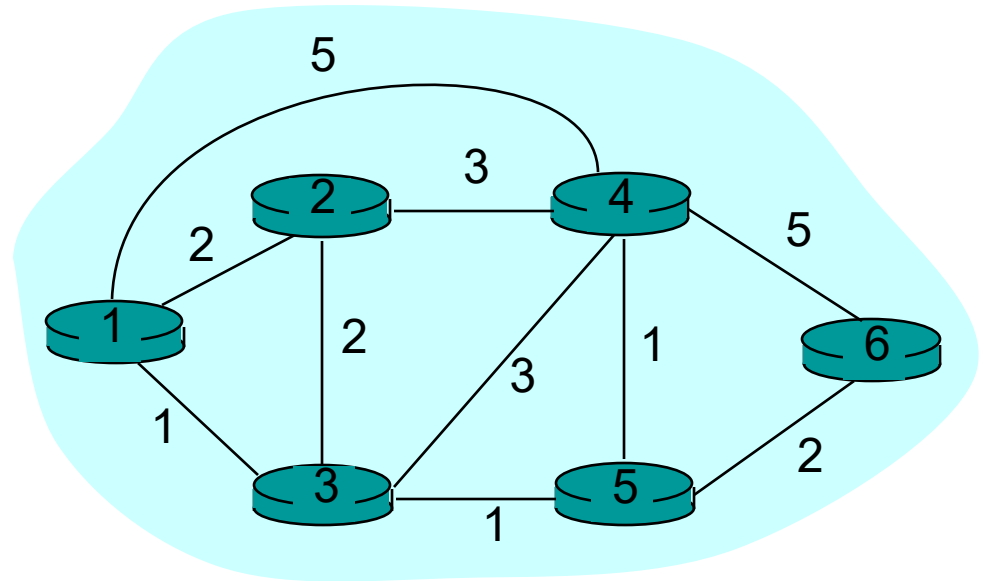
Common Routing Protocols



- Open Shortest Path First (OSPF)
- Intermediate System – Intermediate System (IS-IS)
- Routing Information Protocol (RIP)
- Border Gateway Protocol (BGP)

Modelling a Network

- Modelled as a graph
 - Routers $\Rightarrow$ vertices
 - Links $\Rightarrow$ edges

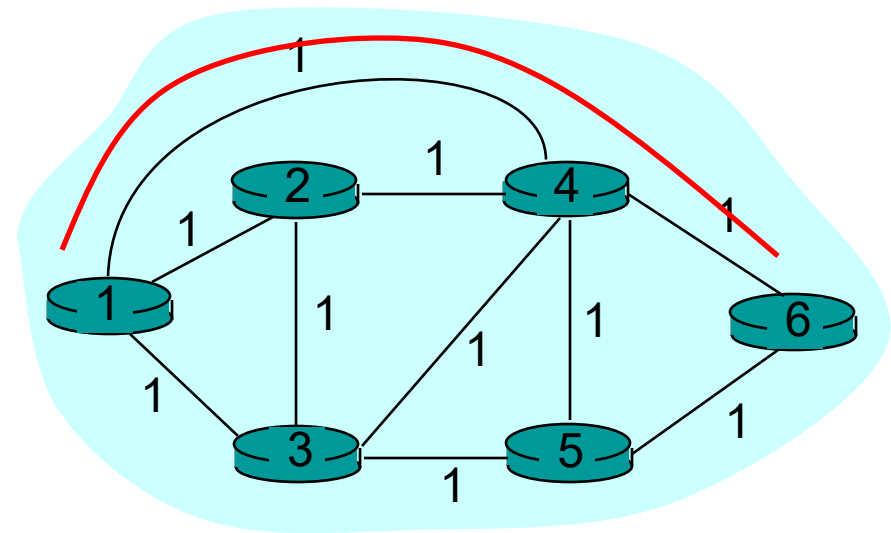
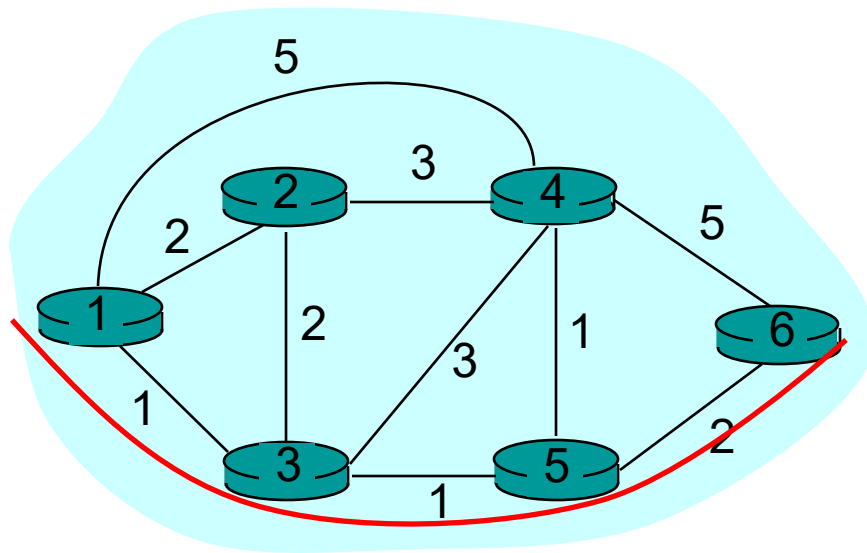


An example network graph

- Edge labels (called metrics) can be interpreted differently
 - as costs, such as the value that is inversely related to the data rate, the current queuing delay

Least-Cost Routing Algorithms

- Find a path with the least cost



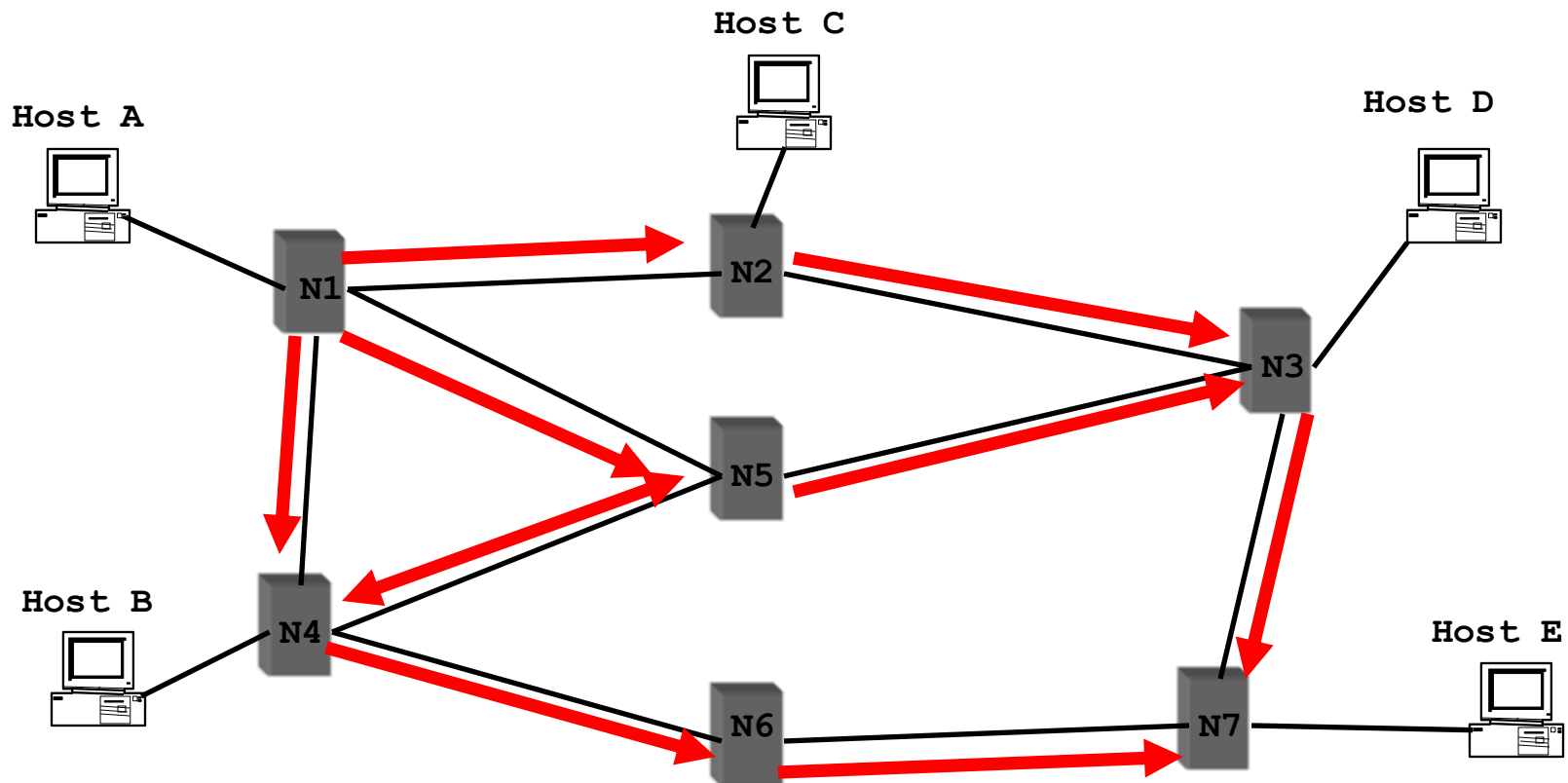
Link State Routing (Dijkstra's Algorithm)

Link State Routing

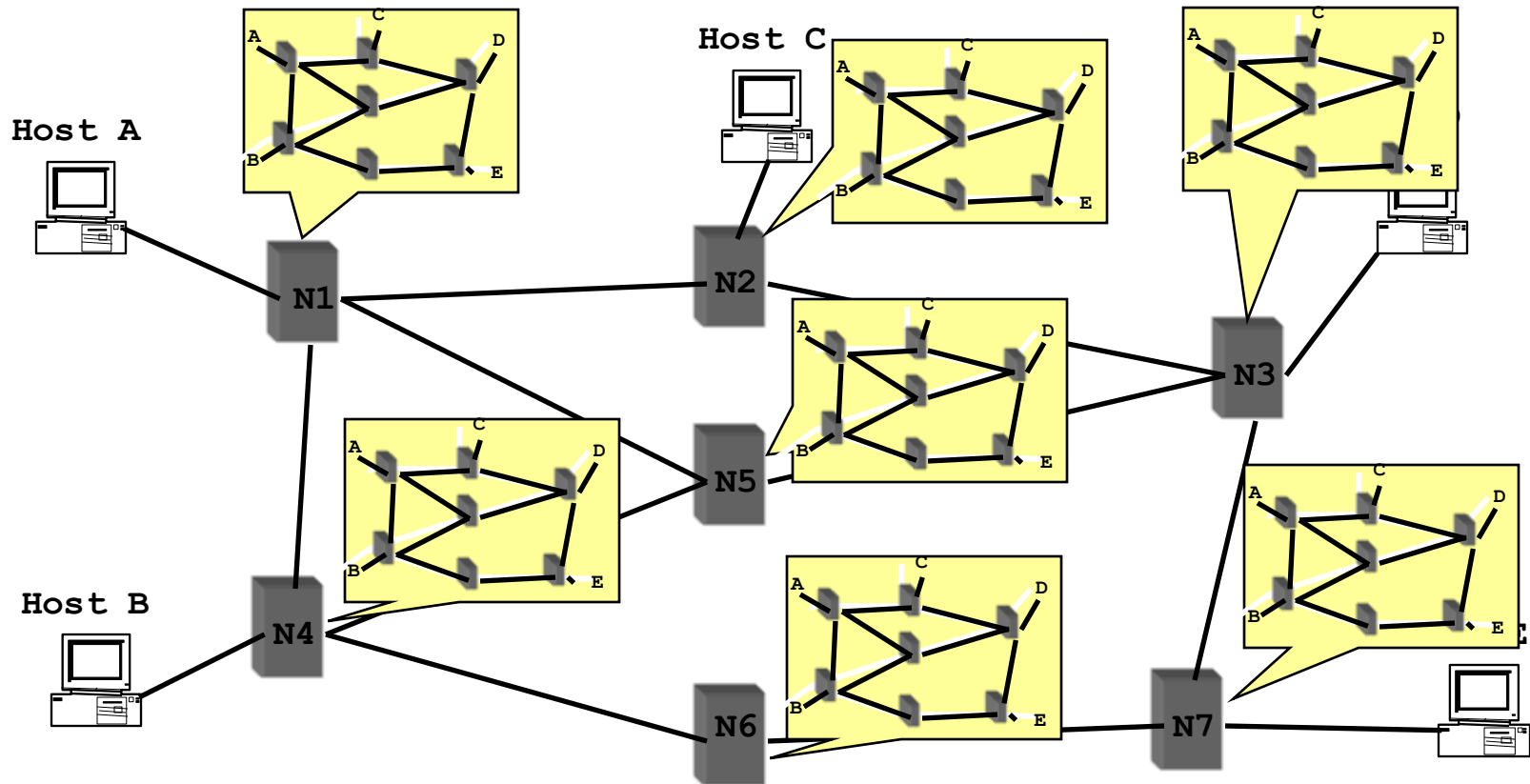
- Each router has the complete network picture
 - Topology
 - Link costs
- How does each router get the global state?
 - Discover its neighbours and learn their network addresses.
 - Set the distance or cost metric to each of its neighbours.
 - Construct a packet telling all it has just learned.
 - Send this packet to and receive packets from all other routers.

Link State Routing: Control Traffic

- Each router reliably **floods** information about its neighbours to every other router

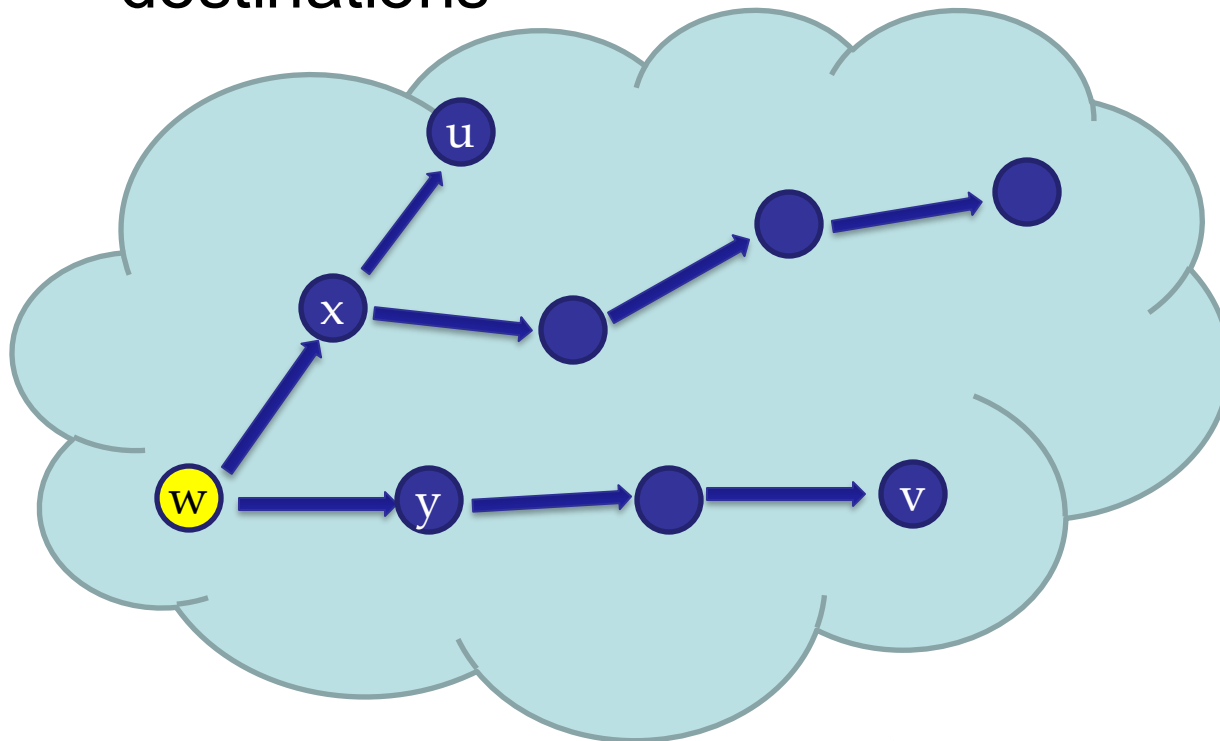


Link State Routing: Node State



Link State Routing: Computing the New Routes

- Each router independently calculates the shortest path from itself to every other router
 - Using, for example, Dijkstra's Algorithm
- Each router constructs a forwarding table for all destinations



Destination	Next hop
u	x
v	y
...	...

Dijkstra's Algorithm

- Basic idea: find the shortest paths from a given source node to all other nodes by developing the paths in order of increasing path length.

Dijkstra's Algorithm (cont.)

- INPUT:

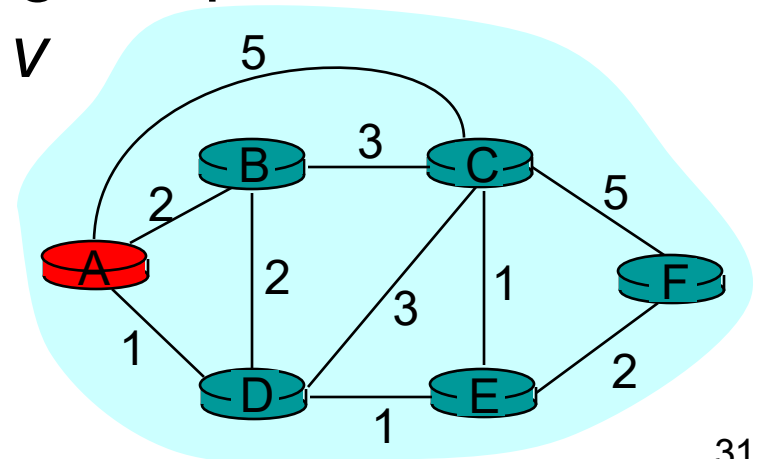
- Network topology (graph), with link costs

- OUTPUT:

- Least cost paths from one node to all other nodes

Dijkstra's Algorithm (cont.)

- **S**: nodes whose least-cost paths already known
 - Initially, **S** = {**u**} where *u* is the source node
- **c(i,j)**: link cost from node *i* to *j*; cost infinite if not direct neighbors; ≥ 0
- **D(v)**: cost of the least-cost path from source *u* to node *v*
- **p(v)**: predecessor node along the path from source *u* to *v*, that is next to *v*



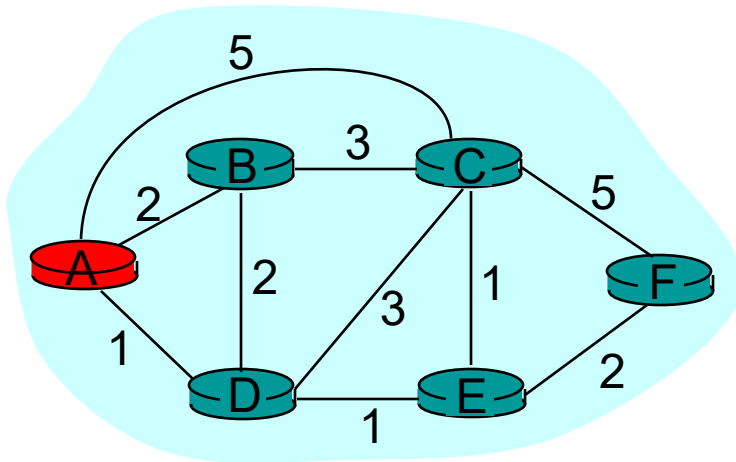
Dijkstra's Algorithm (cont.)

```
1  Initialisation:
2   $S = \{u\}$  /*  $u$  is the source */
3  for all nodes  $v$ 
4    if  $v$  is adjacent to  $u$ 
5       $D(v) = c(u, v)$  /* cost of neighbours known */
6    else  $D(v) = \infty$  /* cost of others unknown */
7
8  Loop:
9    find  $w$  not in  $S$  with the smallest  $D(w)$ 
10   add  $w$  to  $S$ 
11   update  $D(v)$  for all  $v$  adjacent to  $w$  and not in  $S$ :
12   if  $D(w) + c(w, v) < D(v)$  then
        $D(v) = D(w) + c(w, v)$ ;  $p(v) = w$ ;
       /* new cost to  $v$  is either old cost to  $v$  or known
          shortest path cost to  $w$  plus cost from  $w$  to  $v$  */
13  until all nodes in  $S$ 
```



Example: Dijkstra's Algorithm

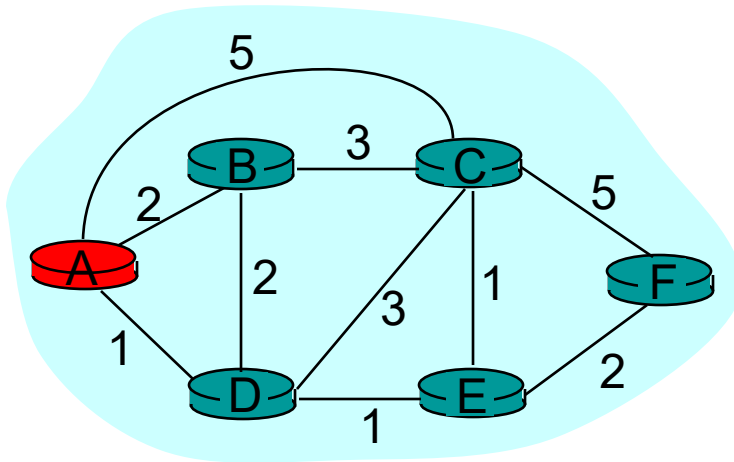
Step	start S	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
0	A					
1						
2						
3						
4						
5						



```
1 Initialisation:
2 S = {A};
3 for all nodes v
4   if v is adjacent to A
5     then  $D(v) = c(A, v)$ ;
6   else  $D(v) = \infty$ ;
...
```

Example: Dijkstra's Algorithm (cont.)

Step	start S	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
0	A	2,A	5,A	1,A	∞	∞
1						
2						
3						
4						
5						

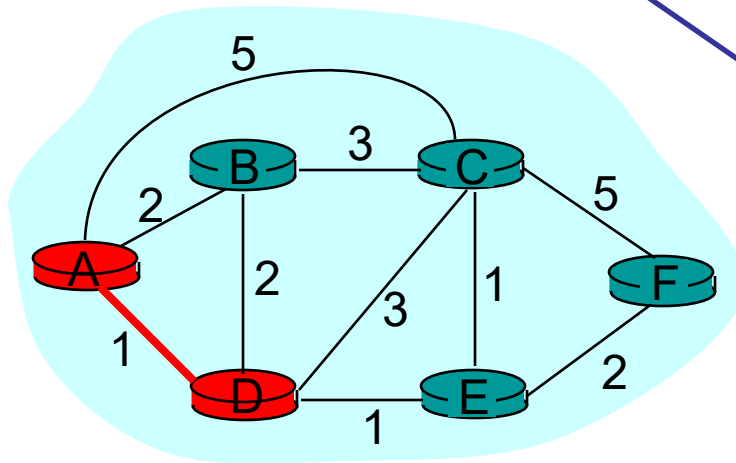


```

...
8  Loop
9  find w not in S s.t. D(w) is a minimum;
10 add w to S;
11 update D(v) for all v adjacent
    to w and not in S:
12 if D(w) + c(w,v) < D(v) then
    D(v) = D(w) + c(w,v); p(v) = w;
13 until all nodes in S;
    
```

Example: Dijkstra's Algorithm (cont.)

Step	start S	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
0	A	2,A	5,A	1,A	∞	∞
1	AD					
2						
3						
4						
5						

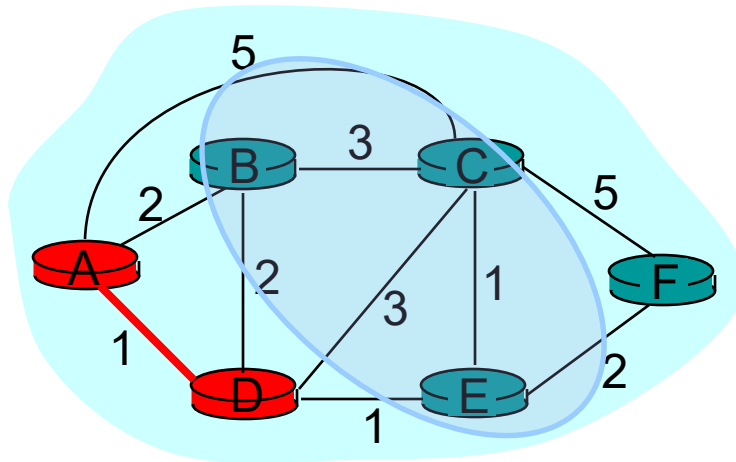


```

...
8  Loop
9  find w not in S s.t. D(w) is a minimum;
10 add w to S;
11 update D(v) for all v adjacent
    to w and not in S:
12 if D(w) + c(w,v) < D(v) then
    D(v) = D(w) + c(w,v); p(v) = w;
13 until all nodes in S;
    
```

Example: Dijkstra's Algorithm (cont.)

Step	start S	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
0	A	2,A	5,A	1,A	∞	∞
1	AD					
2						
3						
4						
5						

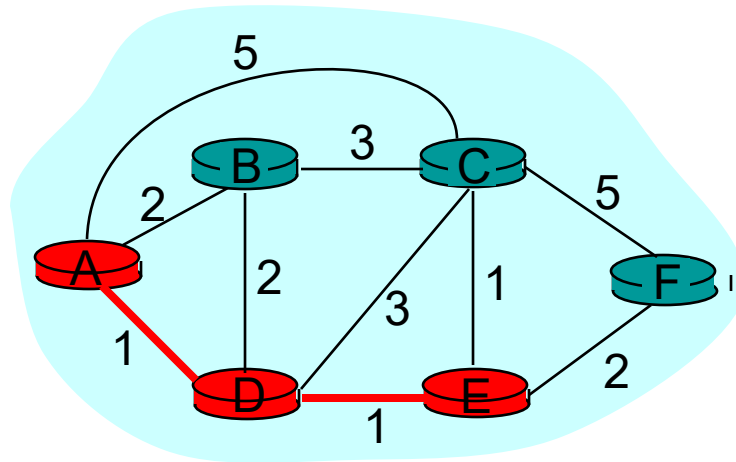


```

...
8  Loop
9  find w not in S s.t. D(w) is a minimum;
10 add w to S;
11 update D(v) for all v adjacent
    to w and not in S:
12 if  $D(w) + c(w,v) < D(v)$  then
     $D(v) = D(w) + c(w,v)$ ;  $p(v) = w$ ;
13 until all nodes in S;
    
```

Example: Dijkstra's Algorithm (cont.)

Step	start S	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
0	A	2,A	5,A	1,A	∞	∞
1	AD		4,D		2,D	∞
2	ADE					
3						
4						
5						

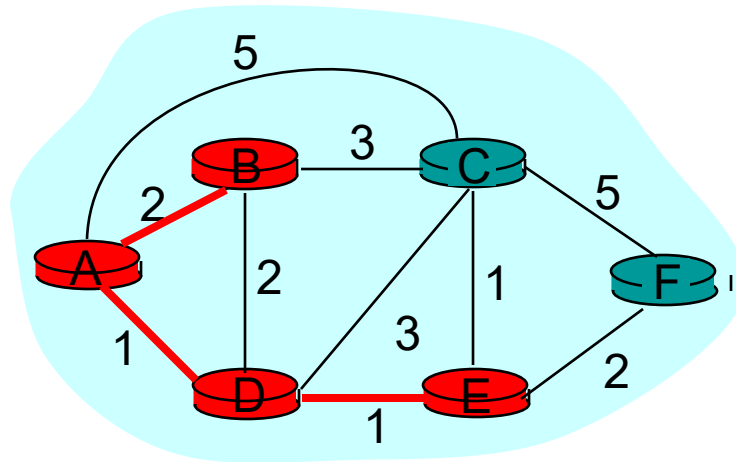


```

...
8  Loop
9  find w not in S s.t. D(w) is a minimum;
10 add w to S;
11 update D(v) for all v adjacent
    to w and not in S:
12   if  $D(w) + c(w,v) < D(v)$  then
         $D(v) = D(w) + c(w,v)$ ;  $p(v) = w$ ;
13 until all nodes in S;
  
```

Example: Dijkstra's Algorithm (cont.)

Step	start S	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
0	A	2,A	5,A	1,A	∞	∞
1	AD		4,D		2,D	∞
2	ADE		3,E			4,E
→ 3	ADEB					
4						
5						

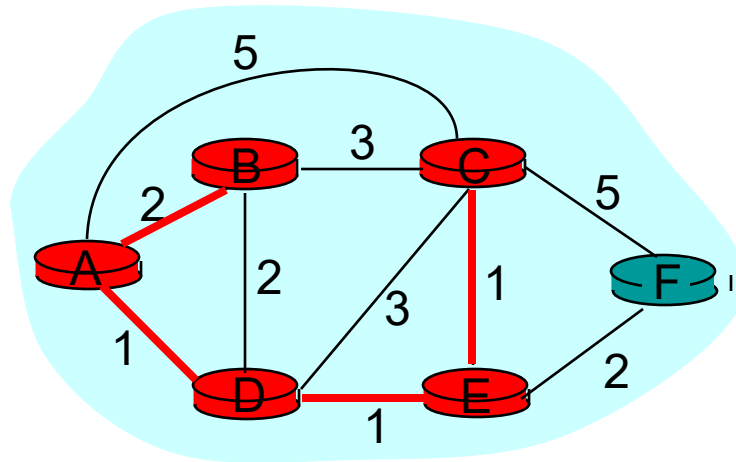


```

...
8  Loop
9  find w not in S s.t. D(w) is a minimum;
10 add w to S;
11 update D(v) for all v adjacent
    to w and not in S:
12   if D(w) + c(w,v) < D(v) then
        D(v) = D(w) + c(w,v); p(v) = w;
13 until all nodes in S;
    
```

Example: Dijkstra's Algorithm (cont.)

Step	start S	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
0	A	2,A	5,A	1,A	∞	∞
1	AD		4,D		2,D	∞
2	ADE		3,E			4,E
3	ADEB					
4	ADEBC					
5						

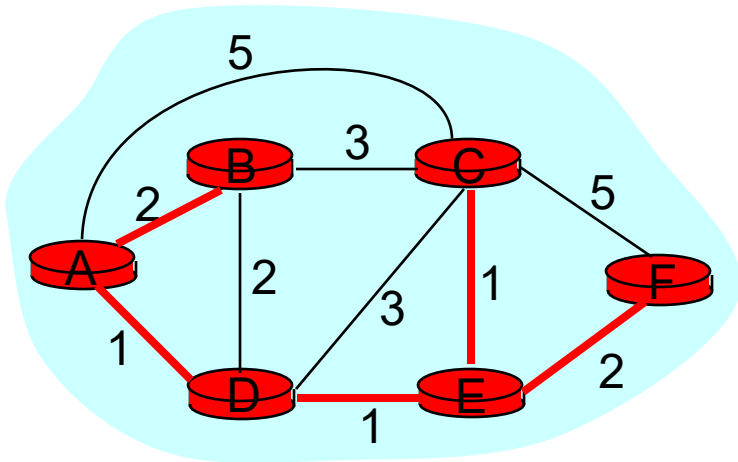


```

...
8  Loop
9  find w not in S s.t. D(w) is a minimum;
10 add w to S;
11 update D(v) for all v adjacent
    to w and not in S:
12   if D(w) + c(w,v) < D(v) then
        D(v) = D(w) + c(w,v); p(v) = w;
13 until all nodes in S;
    
```

Example: Dijkstra's Algorithm (cont.)

Step	start S	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
0	A	2,A	5,A	1,A	∞	∞
1	AD		4,D		2,D	∞
2	ADE		3,E			4,E
3	ADEB					
4	ADEBC					
5	ADEBCF					

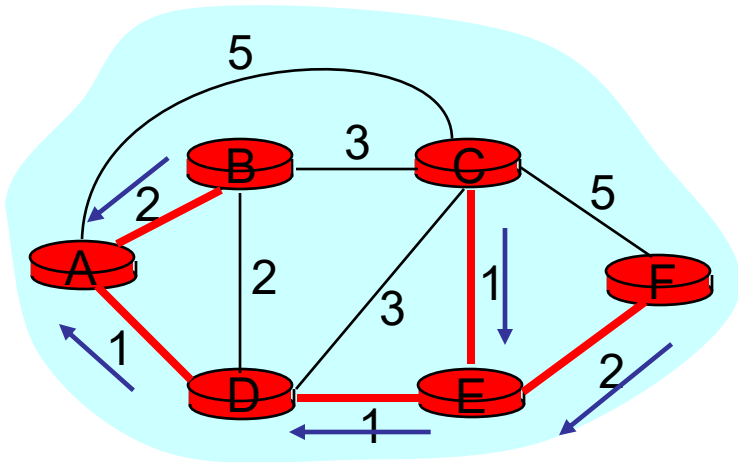


```

...
8  Loop
9  find w not in S s.t. D(w) is a minimum;
10 add w to S;
11 update D(v) for all v adjacent
    to w and not in S:
12   if D(w) + c(w,v) < D(v) then
        D(v) = D(w) + c(w,v); p(v) = w;
13 until all nodes in S;
    
```


Example: Dijkstra's Algorithm (cont.)

Step	start S	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
0	A	2,A	5,A	1,A	∞	∞
1	AD		4,D		2,D	∞
2	ADE		3,E			4,E
3	ADEB					
4	ADEBC					
5	ADEBCF					

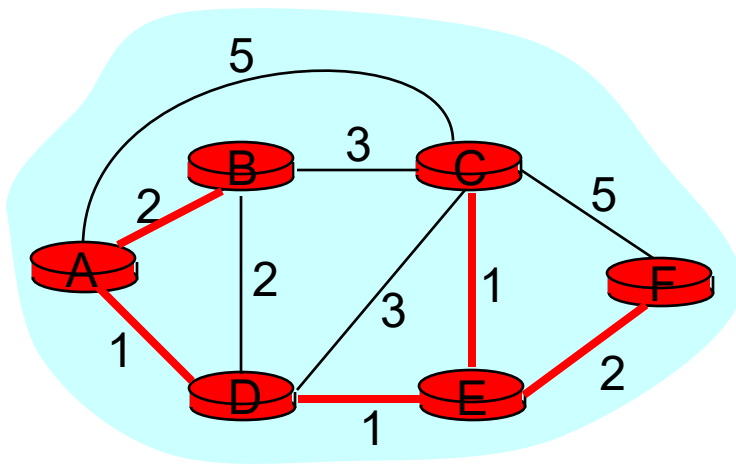


```

...
8  Loop
9  find w not in S s.t. D(w) is a minimum;
10 add w to S;
11 update D(v) for all v adjacent
    to w and not in S:
12 if D(w) + c(w,v) < D(v) then
    D(v) = D(w) + c(w,v); p(v) = w;
13 until all nodes in S;
    
```

The Forwarding Table

- Running Dijkstra at node *A* gives the shortest path from *A* to all destinations
- We then construct the *forwarding* table



Destination	Next hop
B	
C	
D	
E	
F	

When to initiate flooding?

Topology change

- Link or node failure
- Link or node recovery

Configuration change

- Link cost change

Periodically

- Typically, 30 minutes
- Corrects for possible corruption of the data

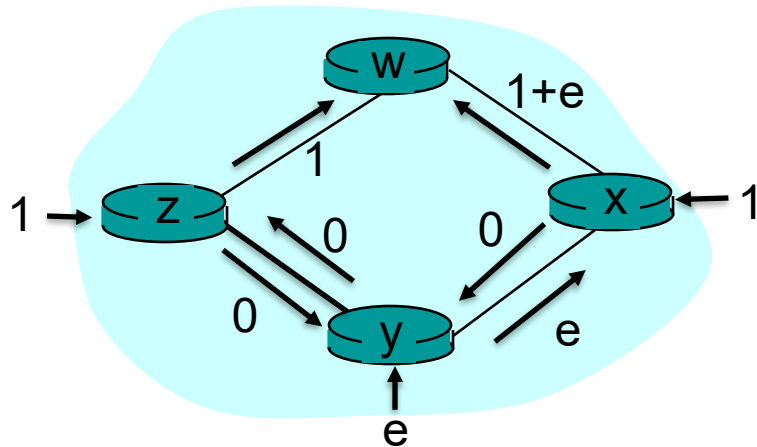
Convergence

- Consistent routing information to all nodes
 - All nodes have the same link-state database
- Consistent forwarding after convergence
 - All nodes have the same link-state database
 - All nodes forward packets on shortest paths

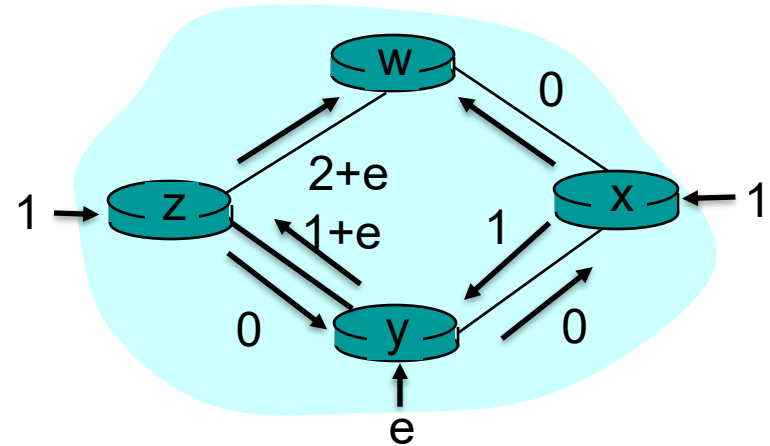
Transient Disruptions

- Detection delay
 - A node does not detect a failed link immediately and forwards data packets into a “blackhole”
 - Depends on timeout for detecting lost “hellos”
- Inconsistent link-state database
 - Some routers know about failure before others
 - The shortest paths are no longer consistent

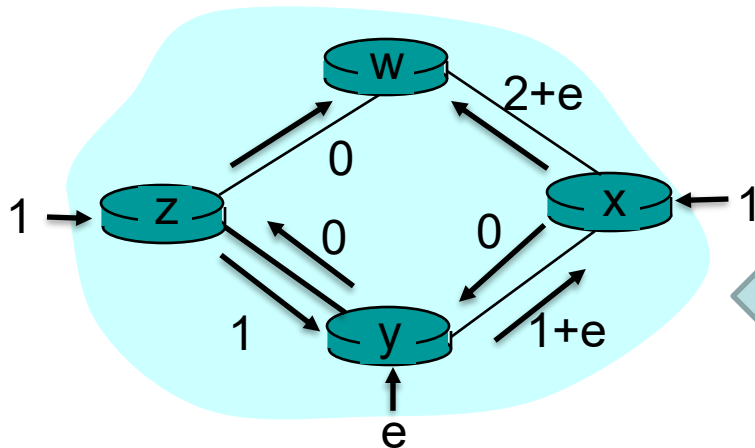
Oscillation with Link State Routing



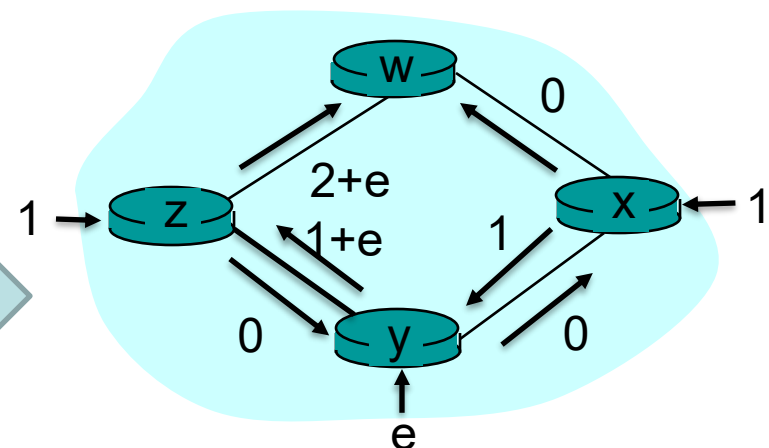
a. Initial routing



b. Recompute



c. Recompute



d. Recompute

Distance Vector Routing (Bellman-Ford Algorithm)

Bellman-Ford Equation

Define

$d_x(y) :=$ cost of least-cost path from x to y

Then

$$d_x(y) = \min_v \{c(x,v) + d_v(y)\}$$

where $\min$ is taken over all neighbors of x .

“After travelling from x to v , if we then take the least-cost path from v to y , the path cost is $c(x,v) + d_v(y)$. Since we must begin by travelling to some neighbor v , the least-cost from x to y is the minimum of $c(x,v) + d_v(y)$ taken over all neighbors v .” from [KR3]

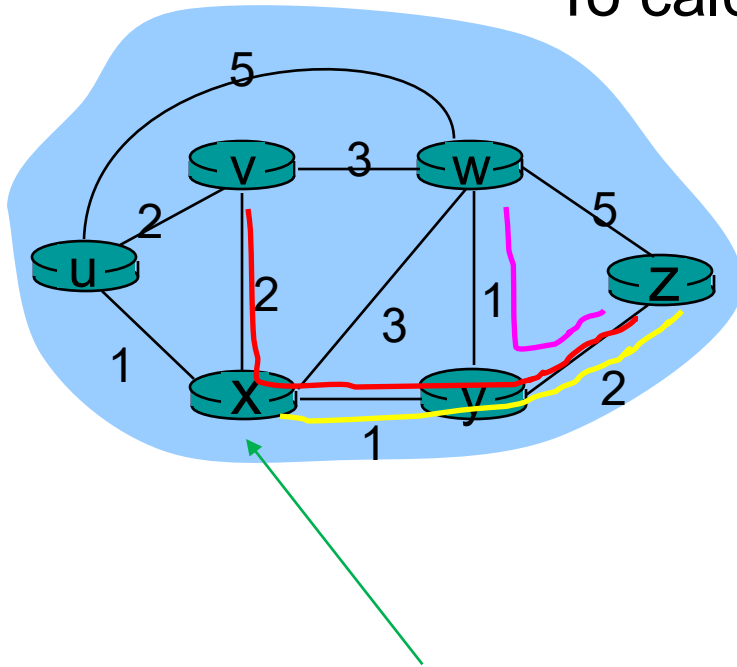
Bellman-Ford example

To calculate $d_u(z)$, according to B-F equation:

$$d_u(z) = \min \{ c(u,v) + d_v(z), \\ \mathbf{c(u,x) + d_x(z)}, \\ c(u,w) + d_w(z) \}$$

Clearly, $d_v(z) = 5$, $d_x(z) = 3$, $d_w(z) = 3$

$$= \min \{ 2 + 5, \\ \mathbf{1 + 3}, \\ 5 + 3 \} = 4$$

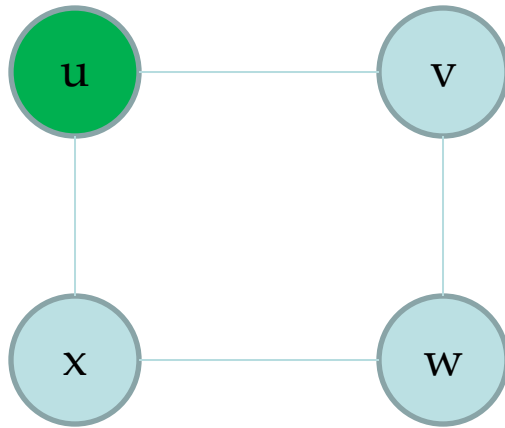


Node that achieves minimum is the next hop in shortest path.

Distance Vector Algorithm

- Estimates:
 - $D_x(y)$ = estimate of least cost from x to y
 - Distance vector: $\mathbf{D}_x = [D_x(y): y \in N]$
- Each node x :
 - Node x knows cost to each neighbor v : $c(x,v)$
 - Node x maintains $\mathbf{D}_x = [D_x(y): y \in N]$
 - Node x also maintains its neighbors' distance vectors
 - For each neighbor v , x maintains $\mathbf{D}_v = [D_v(y): y \in N]$

An illustration



Distance vectors at node u

D_u

D_v

D_x

Distance vector algorithm

Basic idea:

- Each node periodically sends its own distance vector estimate to neighbours
- When a node x receives new DV estimate from neighbor, it updates its own DV using B-F equation:

$$D_x(y) \leftarrow \min_v \{c(x,v) + D_v(y)\} \quad \text{for each node } y \in N$$

- Amazingly, as long as all the nodes continue to exchange their distance vectors in an asynchronous fashion, the estimate $D_x(y)$ converges to the actual least cost $d_x(y)$

Distance Vector Algorithm

Iterative, asynchronous:

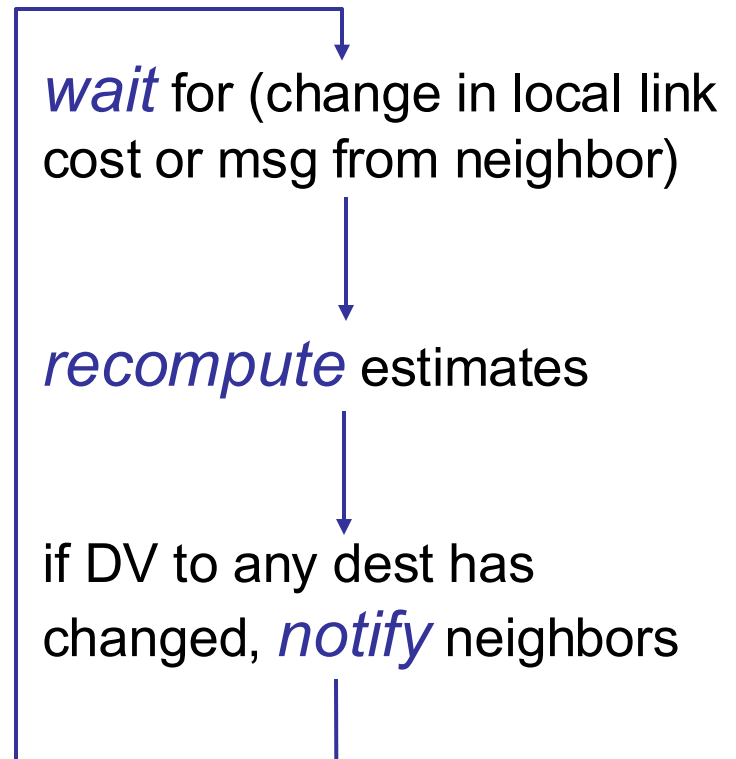
each local iteration caused by:

- local link cost change
- DV update message from neighbour

Distributed:

- each node notifies neighbours *only* when its DV changes
 - neighbours then notify their neighbours if necessary

Each node:



Distance Vector Algorithm

At each node, x:

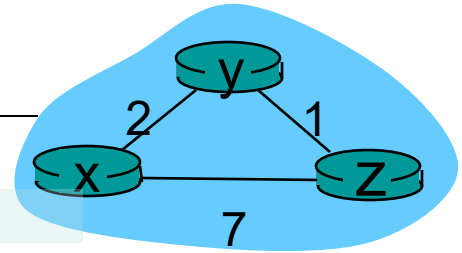
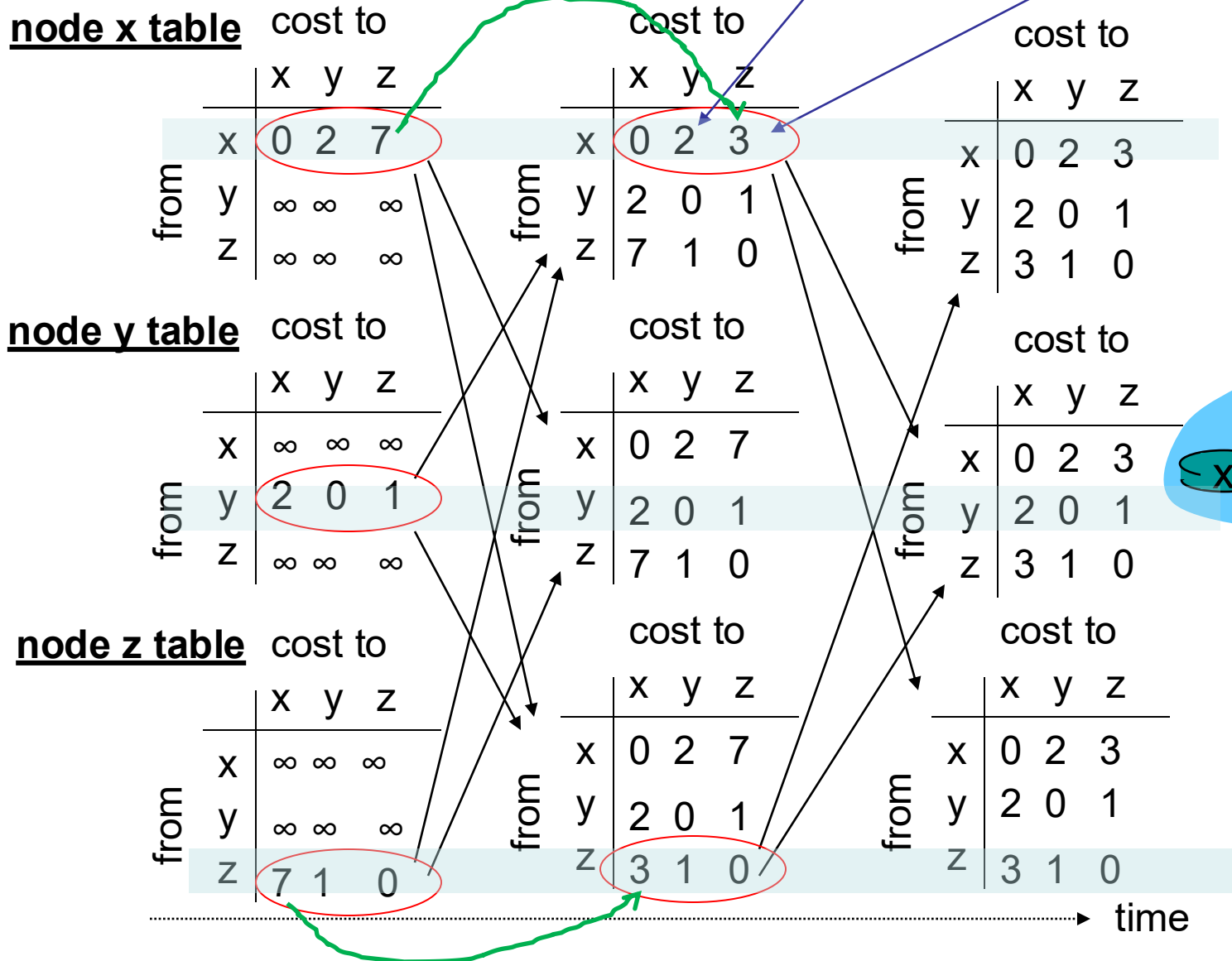
```
1  Initialization:
2      for all destinations y in N:
3           $D_x(y) = c(x,y)$  /*  $c(x,y) = \infty$  if y is not a neighbour */
4      for each neighbour w
5           $D_w(y) = \infty$  for all destinations y in N
6      for each neighbor w
7          send distance vector  $D_x = [D_x(y): y \text{ in } N]$  to w
8  loop
9      wait (until I see a link cost change to some neighbour w
10     or until I receive a distance vector from some neighbour w)
11     for each y in N:
12          $D_x(y) = \min_v \{c(x,v) + D_v(y)\}$  /* v is adjacent to x */
13     if  $D_x(y)$  changed for any destination y
14         send distance vector  $D_x = [D_x(y): y \text{ in } N]$  to all neighbours
15 forever
```

$$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\}$$

$$= \min\{2+0, 7+1\} = 2$$

$$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\}$$

$$= \min\{2+1, 7+0\} = 3$$

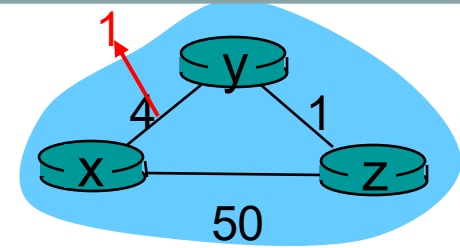


Distance Vector: link cost changes

Link cost changes:

- ❑ node detects local link cost change
- ❑ updates routing info, recalculates distance vector
- ❑ if DV changes, notify neighbors
- ❑ We consider y and z's DVs only here.

$$D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{1+0, 1+5\} = 1;$$



“good
news
travels
fast”

At time t_0 , y detects the link-cost change ($4 \rightarrow 1$), updates its DV, and informs its neighbors.

At time t_1 , z receives the update from y and updates its table. It computes a new least cost to x ($5 \rightarrow 2$) and sends its neighbors its DV.

At time t_2 , y receives z's update and updates its distance table. y's least costs do not change and hence y does *not* send any message to z.

Distance Vector: link cost changes

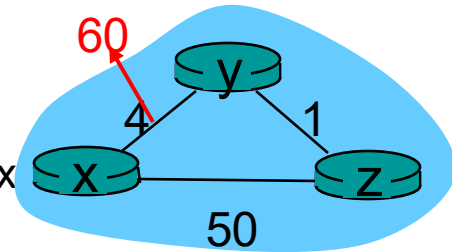
Link cost changes:

Before the link cost changes

- $D_y(x) = 4$, $D_z(x) = 5$ (only y, z's DV entry to dest. x)

At time t0, y detects the link-cost change and re-computes its DV

- $D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{60+0, 1+5\} = 6$;
- The assumption is that y stores its own DV and its neighbours' (and its own link costs to its neighbours).



<i>y's DV table</i>	y	x	z
y	0	4 → 6 (via z)	1
x	4	0	5
z	1	5 (via y)	0

False

y's forwarding table

Dest.	Next-hop
x	z
z	z

z's forwarding table

Dest.	Next-hop
x	y
y	y

Distance Vector: link cost changes

Link cost changes:

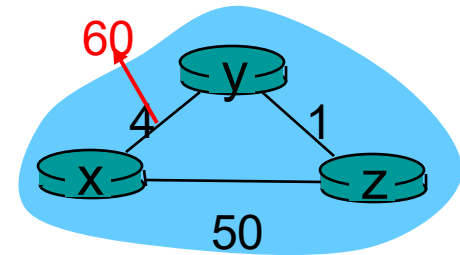
Before the link cost changes

- $D_y(x) = 4$, $D_z(x) = 5$ (y, z's DV entries)

At time t0, y detects the link-cost change and re-compute its DV

- $D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{60+0, 1+5\} = 6$;
- At time t1, y sends its new dv to z; after z receives y's new dv; z can update

$$D_z(x) = \min\{c(z,y) + D_y(x), c(z,x) + D_x(x)\} = \min\{1+6, 50+0\} = 7;$$



<i>z's DV table</i>	y	x	z
y	0	6 (via z)	1
x	51	0	50
z	1	5 → 7 (via y)	0

y's forwarding table

Dest.	Next-hop
x	z
z	z

z's forwarding table

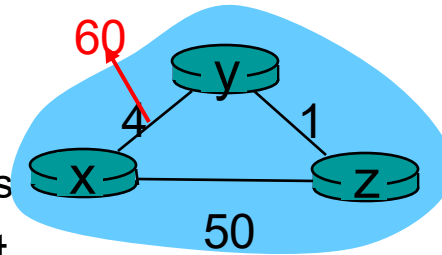
Dest.	Next-hop
x	y
y	y

Distance Vector: link cost changes

Link cost changes:

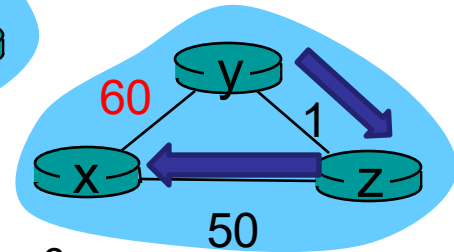
Before the link cost changes

- $D_y(x) = 4$, $D_z(x) = 5$ (y, z's DV entries)



At time t0, y detects the link-cost change and re-computes its DV

- $D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{60+0, 1+5\} = 6$;
- At time t1, y sends its new dv to z; after z receives y's new dv; z can update
 $D_z(x) = \min\{c(z,y) + D_y(x), c(z,x) + D_x(x)\} = \min\{1+6, 50+0\} = 7$;
- At time t2, z sends its new dv to y; similarly y can update
 $D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{60+0, 1+7\} = 8$;
- Then $D_z(x) = \min\{c(z,y) + D_y(x), c(z,x) + D_x(x)\} = \min\{1+8, 50+0\} = 9$;
- $D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{60+0, 1+9\} = 10$;
- ...
- $D_y(x) = \dots = 50$;
- $D_z(x) = \min\{c(z,y) + D_y(x), c(z,x) + D_x(x)\} = \min\{1+50, 50+0\} = 50$;
- $D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{60+0, 1+50\} = 51$;
- $D_z(x) = \min\{c(z,y) + D_y(x), c(z,x) + D_x(x)\} = \min\{1+51, 50+0\} = 50$;



Next-hop changes!

Converged!

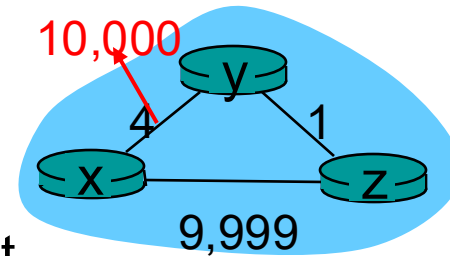
The bad news about the increase in link cost has travelled slowly!

Count-to-infinity problem!

Distance Vector: link cost changes

Link cost changes:

- Before the link cost changes
 - $D_y(x) = 4$, $D_z(x) = 5$
- At time t_0 , y detects the link-cost



Change and re-compute its DV

- $D_y(x): 4 \rightarrow 6 \rightarrow 8 \rightarrow 10, \dots, \rightarrow 9998$;
- $D_z(x): 5 \rightarrow 7 \rightarrow 9 \rightarrow 11, \dots, \rightarrow 9999$; (causing a routing loop!)
- $D_z(x) = \min\{c(z,y) + D_y(x), c(z,x) + D_x(x)\} = \min\{1 + 9998, 9999 + 0\} = 9999$;
- $D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{10000 + 0, 1 + 9999\} = 10000$;
- $D_z(x) = \min\{c(z,y) + D_y(x), c(z,x) + D_x(x)\} = \min\{1 + 10000, 9999 + 0\} = 9999$;

Neighbours exchange distance vectors only!
Distance vectors provide limited information!
z tells y: "I have a path to x with a cost of 7."
It does NOT tell y that this path goes through y!

Distance Vector: link cost changes

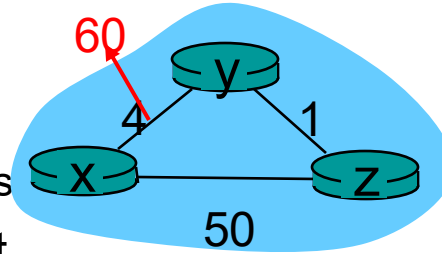
Link cost changes:

Before the link cost changes

- $D_y(x) = 4$, $D_z(x) = 5$ (y, z's DV entries)

At time t_0 , y detects the link-cost change and re-compute its DV

- $D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{60+0, 1+5\} = 6$;
- The assumption is that y stores its own DV and its neighbours' (and its link costs to its neighbours).



<i>y's DV table</i>	y	x	z
y	0	4 → 6 (via z)	1
x	4	0	5
z	1	5 (via y)	0

False

y's forwarding table

Dest.	Next-hop
x	z
z	z

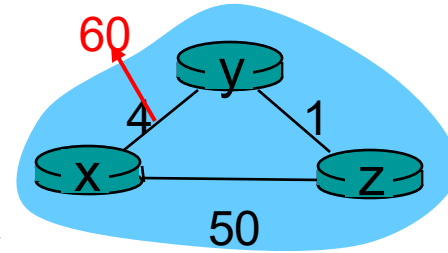
z's forwarding table

Dest.	Next-hop
x	y
y	y

Distance Vector: link cost changes

Poisoned reverse (aka “split horizon with poisoned reverse”):

- ❑ If z routes through y to get to x :
 - z tells y its (z's) distance to x is infinite (so y won't route to x via z)
- ❑ Before the link cost changes
 - $D_y(x) = 4$, $D_z(x) = 5$, but z will lie to y saying “ **$D_z(x) = \infty$** ” (*poisoned reverse*)



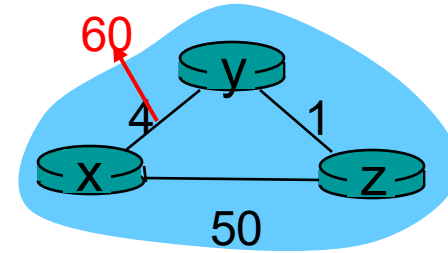
<i>y's DV table</i>	y	x	z
y	0	4	1
x	4	0	∞
z	1	∞	0

Distance Vector: link cost changes

Poisoned reverse:

❑ If z routes through y to get to x :

- z tells y its (z's) distance to x is infinite (so y won't route to x via z)



❑ Before the link cost changes

- $D_y(x) = 4$, $D_z(x) = 5$, but z will lie to y saying “ $D_z(x) = \infty$ ” (*poisoned reverse*)

❑ At time t_0 , y detects the link-cost change and re-compute its dv

- $D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{60+0, 1 + \infty\} = 60$;

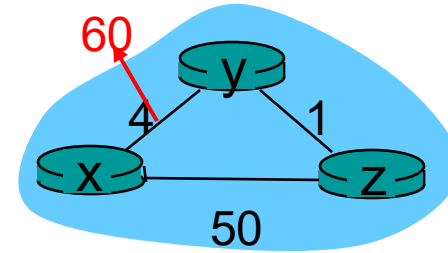
<i>y's DV table</i>	y	x	z
y	0	4 $\rightarrow$ 60 (via x)	1
x	4	0	∞
z	1	∞	0

Distance Vector: link cost changes

Poisoned reverse:

❑ If z routes through y to get to x :

- z tells y its (z's) distance to x is infinite (so y won't route to x via z)



❑ Before the link cost changes

- $D_y(x) = 4$, $D_z(x) = 5$, but z will lie to y saying “ **$D_z(x) = \infty$** ” (*poisoned reverse*)

❑ At time t0, y detects the link-cost change and re-compute its dv

- $D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{60+0, 1 + \infty\} = 60$;
- Now y sends data directly to x;
- At time t1, y sends its new dv to z; after z receives y's new dv; z can update $D_z(x) = \min\{c(z,y) + D_y(x), c(z,x) + D_x(x)\} = \min\{1+60, 50+0\} = 50$;

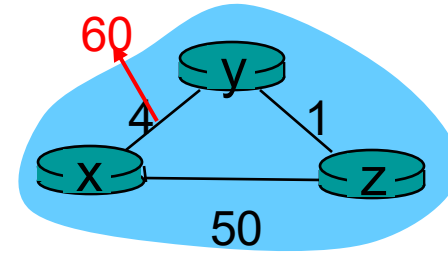
<i>z's DV table</i>	y	x	z
y	0	60	∞
x	∞	0	∞
z	1	50 (via x)	0

Distance Vector: link cost changes

Poisoned reverse:

□ If z routes through y to get to x :

- z tells y its (z's) distance to x is infinite (so y won't route to x via z)

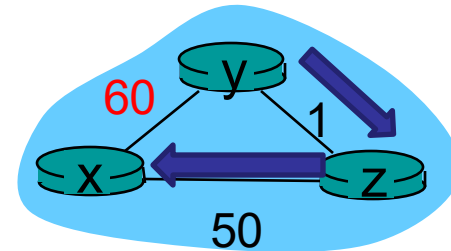


□ Before the link cost changes

- $D_y(x) = 4$, $D_z(x) = 5$, but z will lie to y saying “ $D_z(x) = \infty$ ” (*poisoned reverse*)

□ At time t0, y detects the link-cost change and re-compute its dv

- $D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{60+0, 1 + \infty\} = 60$;
- Now y sends data directly to x;



- At time t1, y sends its new dv to z; after z receives y's new dv; z can update

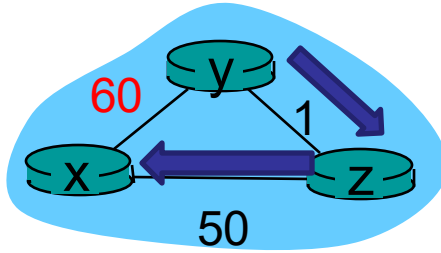
$$D_z(x) = \min\{c(z,y) + D_y(x), c(z,x) + D_x(x)\} = \min\{1+60, 50+0\} = 50;$$

- At time t2, z sends its new dv to y without lying since it will not route through y; similarly y can update

$$D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{60+0, 1+50\} = 51;$$

- Then $D_z(x) = \min\{c(z,y) + D_y(x), c(z,x) + D_x(x)\} = \min\{1 + \infty, 50+0\} = 50$; y lies to z this time because it routes through z;

It seems to be effective!

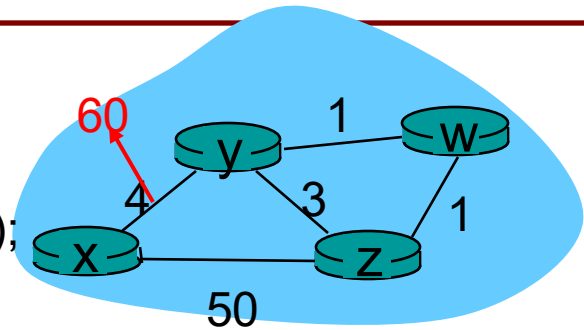


y uses z as its next-hop and will lie to z by saying that “I have a path to x with a cost of *infinity*.”
(Do not count on me to route your traffic to x, -- poisoned reverse.)

- **Poisoned reverse does not solve the general count-to-infinity problem.**
- Loops involving three or more nodes (rather than simply two immediately neighbouring nodes) will not be detected by the poisoned reverse technique.

A loop involving 3 nodes

Consider y,z,w distance table entries to x only. *Using poisoned reverse*,
z (tells) $\rightarrow$ w, $D_z(x) = \infty$; z $\rightarrow$ y $D_z(x) = 6$ (not lying);
w $\rightarrow$ y, $D_w(x) = \infty$; w $\rightarrow$ z $D_w(x) = 5$ (not lying);
y $\rightarrow$ w, $D_y(x) = 4$ (not lying); y $\rightarrow$ z $D_y(x) = 4$ (not lying);
Then there is link-cost change (4 $\rightarrow$ 60);



A loop involving 3 nodes

Consider y,z,w distance table entries to

x only. Using poisoned reverse,

z (tells) $\rightarrow$ w, $D_z(x) = \infty$; $z \rightarrow y$ $D_z(x) = 6$ (not lying);

$w \rightarrow y$, $D_w(x) = \infty$; $w \rightarrow z$ $D_w(x) = 5$ (not lying);

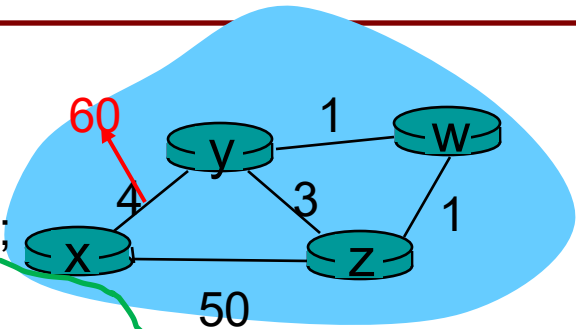
$y \rightarrow w$, $D_y(x) = 4$ (not lying); $y \rightarrow z$ $D_y(x) = 4$ (not lying);

Then there is link-cost change ($4 \rightarrow 60$);

At t1, y updates its $D_y(x) = 9$ (via z, z did not lie:);

Now y notifies w and z; $y \rightarrow w$, $D_y(x) = 9$; $y \rightarrow z$ $D_y(x) = \infty$;

$D_y(x) = \min\{c(y,z)+D_z(x), c(y,w) + D_w(x), c(y,x) + D_x(x)\} = \min\{3+6, 1+ \infty, 60+0\} = 9$ (via z)



A loop involving 3 nodes

Consider y,z,w distance table entries to

x only. Using poisoned reverse,

$z \rightarrow w$, $D_z(x) = \infty$; $z \rightarrow y$ $D_z(x) = 6$ (not lying);

$w \rightarrow y$, $D_w(x) = \infty$; $w \rightarrow z$ $D_w(x) = 5$ (not lying);

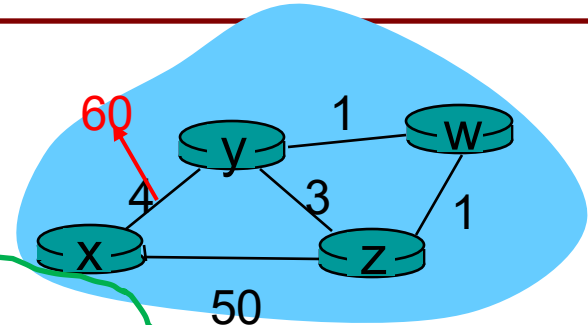
$y \rightarrow w$, $D_y(x) = 4$ (not lying); $y \rightarrow z$ $D_y(x) = 4$ (not lying);

Then there is link-cost change ($4 \rightarrow 60$);

At t_1 , y updates its $D_y(x) = 9$ (via z, z did not lie:);

Now y notifies w and z; $y \rightarrow w$, $D_y(x) = 9$; $y \rightarrow z$ $D_y(x) = \infty$;

$$D_y(x) = \min\{c(y,z)+D_z(x), c(y,w) + D_w(x), c(y,x) + D_x(x)\} = \min\{3+6, 1+ \infty, 60+0\} = 9 \text{ (via z)}$$



w re-computes:

$$D_w(x) = \min\{c(w,y) + D_y(x), c(w,z) + D_z(x)\} \\ = \min\{1+9, 1+ \infty\} = 10 \text{ (via y);}$$

w notifies y and z; $\rightarrow y$ $D_w(x) = \infty$ (*lying*);

$\rightarrow z$ $D_w(x) = 10$ (telling truth);

A loop involving 3 nodes

Consider y,z,w distance table entries to

x only. Using poisoned reverse,

$z \rightarrow w$, $D_z(x) = \infty$; $z \rightarrow y$ $D_z(x) = 6$ (not lying);

$w \rightarrow y$, $D_w(x) = \infty$; $w \rightarrow z$ $D_w(x) = 5$ (not lying);

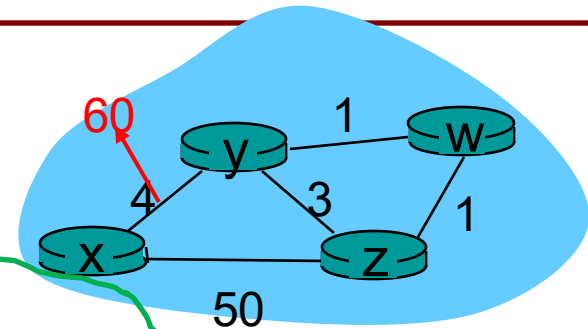
$y \rightarrow w$, $D_y(x) = 4$ (not lying); $y \rightarrow z$ $D_y(x) = 4$ (not lying);

Then there is link-cost change ($4 \rightarrow 60$);

At t_1 , y updates its $D_y(x) = 9$ (via z, z did not lie:);

Now y notifies w and z; $y \rightarrow w$, $D_y(x) = 9$; $y \rightarrow z$ $D_y(x) = \infty$;

$$D_y(x) = \min\{c(y,z)+D_z(x), c(y,w) + D_w(x), c(y,x) + D_x(x)\} = \min\{3+6, 1+ \infty, 60+0\} = 9 \text{ (via z)}$$



w re-computes:

$$D_w(x) = \min\{c(w,y) + D_y(x), c(w,z) + D_z(x)\} \\ = \min\{1+9, 1+ \infty\} = 10 \text{ (via y);}$$

w notifies y and z; $\rightarrow y$ $D_w(x) = \infty$ (*lying*);
 $\rightarrow z$ $D_w(x) = 10$ (telling truth);

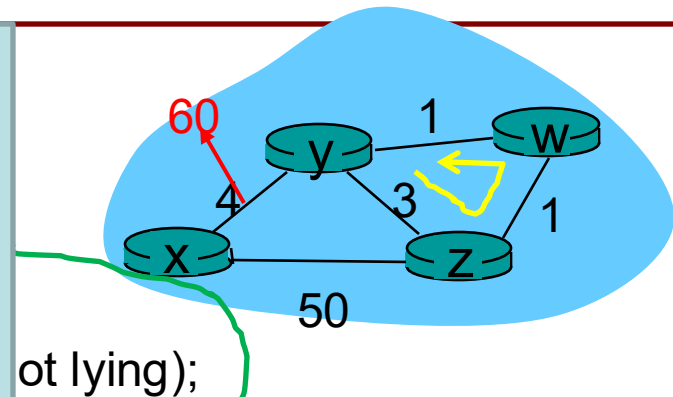
z re-computes:

$$D_z(x) = \min\{c(z,w) + D_w(x), c(z,y) + D_y(x), \\ c(z,x) + D_x(x)\} = \min\{1+10, 3+ \infty, 50\} = 11 \\ \text{(via w);}$$

z notifies w and y; $\rightarrow w$ $D_z(x) = \infty$ (*lying*);
 $\rightarrow y$ $D_z(x) = 11$ (telling truth);

A loop involving 3 nodes

Co y re-computes:
 $D_y(x) = \min\{c(y,z) + D_z(x), c(y,w) + D_w(x), c(y,x) + D_x(x)\} = \min\{3+11, 1+\infty, 60\} = 14$
 (via z);
 w y notifies z and w; $\rightarrow_z D_y(x) = \infty$ (*lying*);
 $\rightarrow_w D_y(x) = 14$ (telling truth);
 Th



At t1, y updates its $D_y(x) = 9$ (via z, z did not lie:);

Now y notifies w and z; $y \rightarrow w, D_y(x) = 9$; $y \rightarrow z D_y(x) = \infty$;

$$D_y(x) = \min\{c(y,z) + D_z(x), c(y,w) + D_w(x), c(y,x) + D_x(x)\} = \min\{3+0, \infty, 60+0\} = 3 \text{ (via z)}$$

w re-computes:
 $D_w(x) = \min\{c(w,y) + D_y(x), c(w,z) + D_z(x)\} = \min\{1+9, 1+\infty\} = 10$ (via y);
 w notifies y and z; $\rightarrow_y D_w(x) = \infty$ (*lying*);
 $\rightarrow_z D_w(x) = 10$ (telling truth);

z re-computes:
 $D_z(x) = \min\{c(z,w) + D_w(x), c(z,y) + D_y(x), c(z,x) + D_x(x)\} = \min\{1+10, 3+\infty, 50\} = 11$
 (via w);
 z notifies w and y; $\rightarrow_w D_z(x) = \infty$ (*lying*);
 $\rightarrow_y D_z(x) = 11$ (telling truth);

A loop involving 3 nodes

Consider y,z,w distance table entries to

x only. Using poisoned reverse,

$z \rightarrow w$, $D_z(x) = \infty$; $z \rightarrow y$ $D_z(x) = 6$ (not lying);

$w \rightarrow y$, $D_w(x) = \infty$; $w \rightarrow z$ $D_w(x) = 5$ (not lying);

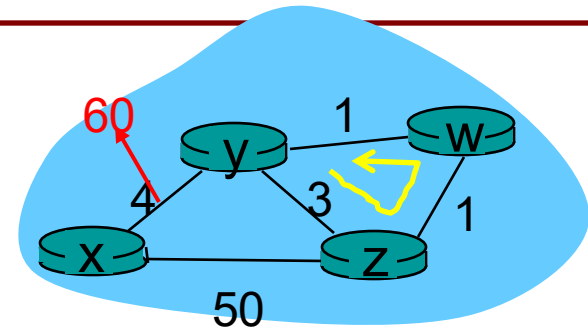
$y \rightarrow w$, $D_y(x) = 4$ (not lying); $y \rightarrow z$ $D_y(x) = 4$ (not lying);

Then there is link-cost change (4 $\rightarrow$ 60);

At t1, y updates its $D_y(x) = 9$ (via z, z did not lie:);

Now y notifies w and z; $y \rightarrow w$, $D_y(x) = 9$; $y \rightarrow z$ $D_y(x) = \infty$;

$$D_y(x) = \min\{c(y,z)+D_z(x), c(y,w) + D_w(x), c(y,x) + D_x(x)\} = \min\{3+6, 1+ \infty, 60+0\} = 9 \text{ (via z)}$$



	t0	t1	t2	t3	t4
z	$\rightarrow w$, $D_z(x) = \infty$; $\rightarrow y$, $D_z(x) = 6$;			$\rightarrow w$, $D_z(x) = \infty$; $\rightarrow y$, $D_z(x) = 11$;	
w	$\rightarrow y$, $D_w(x) = \infty$; $\rightarrow z$ $D_w(x) = 5$;		$\rightarrow y$, $D_w(x) = \infty$; $\rightarrow z$, $D_w(x) = 10$;		
y	$\rightarrow w$, $D_y(x) = 4$; $\rightarrow z$, $D_y(x) = 4$;	$\rightarrow w$, $D_y(x) = 9$; $\rightarrow z$, $D_y(x) = \infty$;			$\rightarrow w$, $D_y(x) = 14$; $\rightarrow z$ $D_y(x) = \infty$;

This sequence (y-w-z) repeats and a routing loop exits.

Distance Vector Algorithm

Iterative, asynchronous:

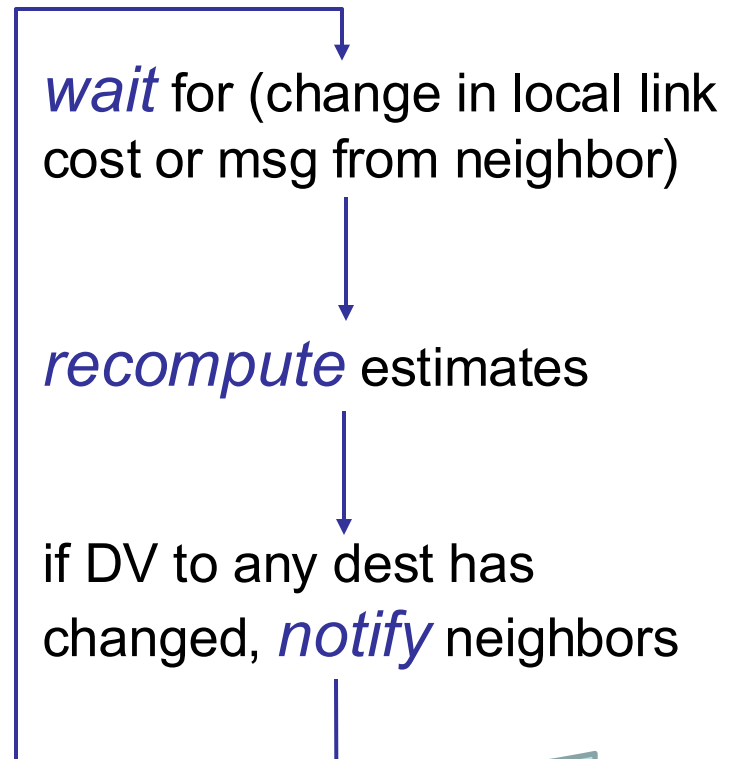
each local iteration caused by:

- local link cost change
- DV update message from neighbour

Distributed:

- each node notifies neighbours *only* when its DV changes
 - neighbours then notify their neighbours if necessary

Each node:



DV has the count-to-infinity problem and poisoned reverse does not solve it. *In RIP the maximum cost of a path is limited to 15.*

Summary

Classification of routing algorithms

LS a **global** routing algorithm

- compute the least-cost path between a source and destination using complete and global knowledge about the network

DV a **decentralised** routing algorithm

- calculate the least-cost path in an iterative and distributed manner

Both **dynamic** routing algorithms

- more responsive to network changes but more susceptible to problems such as routing loops and oscillation in routes

Message complexity

- LS: $O(nm)$ with n nodes and m links
- DV: message exchange between neighbours only at each iteration

Speed of convergence

- LS: $O(n^2)$
- DV: convergence time varies; the count-to-infinity problem

Robustness: what happens if router malfunctions?

LS

- a node can advertise incorrect *link* cost
- each node computes only its *own* table

DV

- a node can advertise incorrect *path* cost
- each node's distance vector is used by others
- error propagates through the entire network

Routing Algorithms and Routing Protocols in the Internet

Intra-AS Routing

Routing Protocols

RIP

OSPF

BGP

Routing Algorithms

Bellman-Ford (Distance-vector) Algorithm

Dijkstra's Algorithm

A variant of Bellman-Ford Algorithm (path-vector)

Inter-AS Routing

References

- William Stallings, Data and Computer Communications, Pearson, 10th edition, 2013.
 - Chapter 19 Routing (19.1, 19.3, 19.4)
- Andrew S. Tanenbaum & David J. Wetherall, Computer Networks, Prentice Hall, 5th edition, 2010.
 - Chapter 5 The Network Layer (5.1, 5.2)
- James F. Kurose & Keith W. Ross, Computer networking: a top-down approach featuring the Internet, Addison Wesley, 3rd edition, 2005.
 - Chapter 4 The Network Layer (4.5)

Acknowledgements

- Lecture notes are developed based on slides from the following three sources:
 - Dr Mengmeng Ge's lecture notes for COSC264, University of Canterbury, 2023
 - Assoc Prof Dan Kim's lecture notes for COSC264, University of Canterbury, 2017
 - Prof Aleksandar Kuzmanovic's lecture notes for CS340, Northwestern University, 2005, https://users.cs.northwestern.edu/~akuzma/classes/CS340-w05/lecture_notes.htm